Lahore University of Management Sciences

AI651 — Deep Learning for Space, Time and Graphs
Fall 2026

Assignment 2

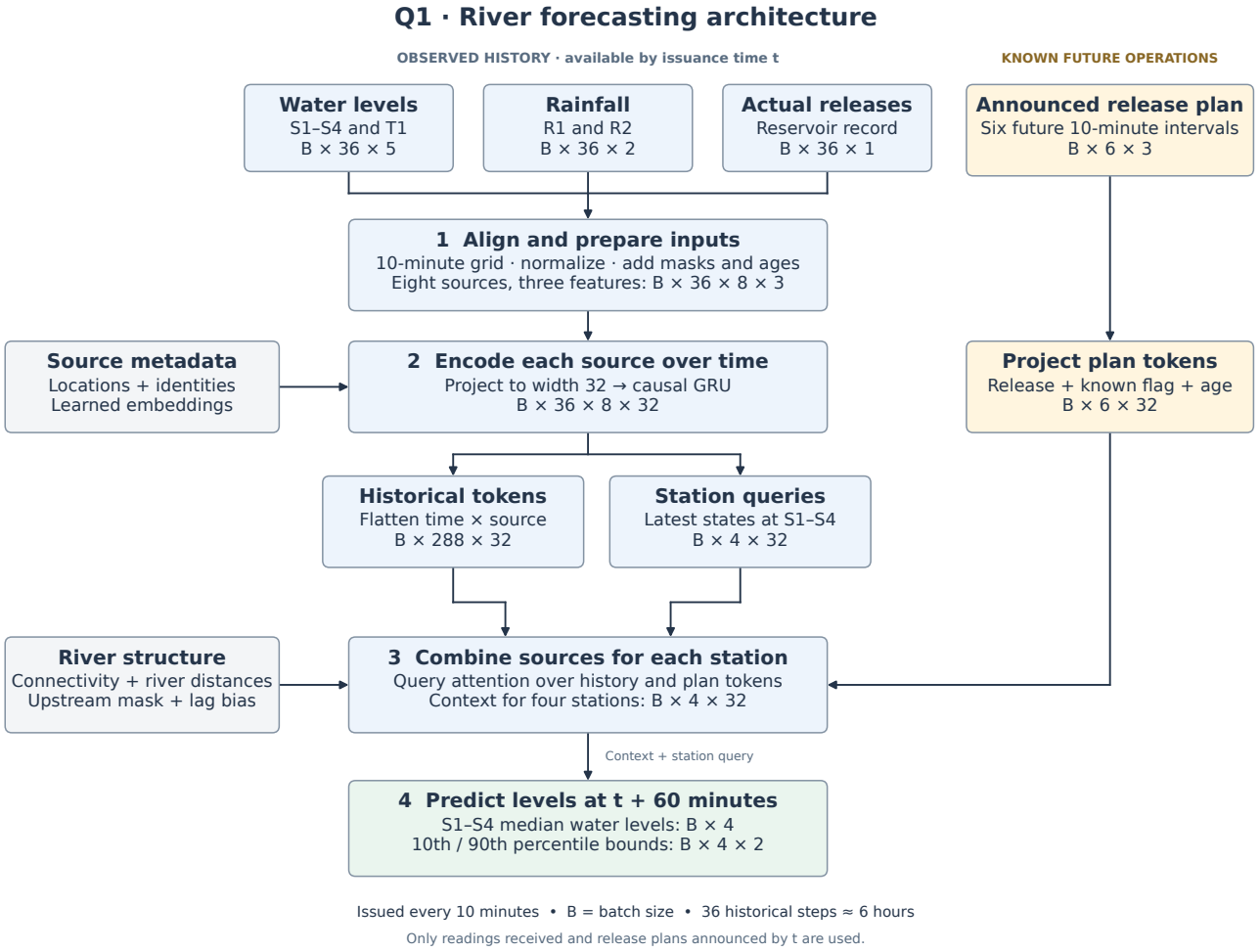
Muhammad Zaid Iftikhar
25280039

Code repository:
github.com/zic112/motion-and-river-forecasting

Experimental source: `Question2_normalized/Question2_final.ipynb`.

Question 1 — River Forecasting Proposal

1. Architecture diagram.



B is batch size. The mentioned eight historical sources are level gauges (5), rain gauges (2) and reservoir releases. Future data is not part of the observed history.

2. Assumptions.

The topology, data sources and forecast schedule are supplied facts. The design assumes:

- **A1 — Records:** At least two years of past data exists including seasonal changes and some floods. This allows the model to learn seasonal trends as well.
- **A2 — Data quality:** Timestamps and units can be reconciled. Missing or delayed readings are handled by masks and observation ages. Gauge datums must be documented; arrival logs or conservative delay assumptions reproduce historical availability.
- **A3 — History:** six hours are enough to capture useful short-term responses. We can extend it if validation supports a longer window.
- **A4 — Dynamics:** The main impact is downstream and it varies in time. This motivates upstream input paths with learned lags; strong backwater would require extra connections.
- **A5 — Unmeasured inflows:** Some of the water flow is unmeasured. Its uncertain effect on water levels matters especially at S3–S4; the model does not separately forecast tributary discharge.
- **A6 — Plans:** Release records identify effective times. Use historical plans only if they provide valid context and were announced by issuance. Otherwise mark them unavailable.

3. Forecasting task and data specification.

Every ten minutes, predict S1–S4 levels at $t + 60$ **minutes** using 36 positions, $t - 350, t - 340, \ldots, t$. Inputs must arrive by issuance t; future levels are targets only.

Input/output	Dimensions and axis meanings	Time coverage / availability
Levels at S1–S4 and T1	$B \times 36 \times 5$: examples, times, gauges	Historical grid through t; received by t
Rainfall at R1/R2	$B \times 36 \times 2$: examples, times, gauges	Ten-minute rainfall totals ending at each grid time; received by t
Actual reservoir release	$B \times 36 \times 1$: examples, times, release source	Historical interval-average release, received by t
Historical availability and age	Each $B \times 36 \times 8$: examples, times, sources	Known-value flag and elapsed time since the last usable observation
Future release plan	$B \times 6 \times 3$: examples, future intervals, features	Release, known flag and announcement age for intervals ending $t + 10, \ldots, t + 60$; announcements no later than t
Source locations and identities	8×2 coordinates and eight categorical identities; shared across batch	Static metadata; coordinates projected into a common metric system
River connectivity and distances	$6 \times 6 \times 2$: receiving/source river nodes, ancestry flag and river distance	Six nodes: reservoir, S1–S4, T1; shared across batch; distances used for connected pairs
Forecast and training target	Each $B \times 4$: examples, ordered stations S1–S4	Water levels at $t + 60$, in each station's documented datum
Uncertainty bounds	$B \times 4 \times 2$: examples, stations, lower/upper bounds	Proposed 10th and 90th percentiles at $t + 60$

Timestamps are to be aligned into ten-minute intervals. Carry forward the latest available reading with age and imputation flags. If no earlier reading exists, use a training-set reference marked unavailable. Post-issuance information will be excluded and normalization will be fit on training data only. Unknown plans do not mean zero release.

4. Model computations and design justification.

For source i at step s, $x_{i,s}$ contains its normalized value, availability and age. With source/location embedding e_i and activation ϕ:

$$u_{i,s} = \phi(W_x x_{i,s} + e_i)$$
$$h_{i,s} = \mathrm{GRU}(u_{i,s}, h_{i,s-1}).$$

The causal GRU holds past patterns. Reshape its 36×8 states into 288 width-32 tokens and append six projected plan tokens (diagram).

Form station query q_j from its latest hidden state and identity. For token v_a in allowed set $\mathcal{A}_j$:

$$e_{j,a} = \frac{(W_Q q_j)^\top W_K v_a}{\sqrt{32}} + b_\theta(\Delta t_a, d_{ja}, \text{source type}, \text{location offset})$$
$$\alpha_{j,a} = \mathrm{softmax}_{a\in\mathcal{A}_j}(e_{j,a})$$
$$z_j = \sum_{a\in\mathcal{A}_j} \alpha_{j,a} W_V v_a.$$

Here, Δt_a is relative time and d_{ja} is river distance. Rainfall uses relative locations and source identity. Training learns W_Q, W_K, W_V and b_θ. Condition-dependent attention handles unequal spacing and variable lags (A3–A4); its weights are not measured travel times.

Each station uses its own history, rainfall and also uses the following data:

- **S1:** reservoir history and announced plans.
- **S2:** S1, reservoir inputs and T1, whose tributary joins before S2.
- **S3:** S1/S2/T1 and reservoir inputs. Ungauged tributary effects remain partly unknown.
- **S4:** all upstream river sources and reservoir inputs.

Unknown catchment boundaries justify access to both rain gauges. Direct ancestor connections reach distant upstream data without many graph layers. Connectivity restricts paths; training learns responses. Flow conservation cannot apply directly to water levels, which are not discharge.

The output head maps $[q_j, z_j]$ to median p_j and ordered bounds:

$$\widehat{y}_{j,0.5} = p_j$$
$$\widehat{y}_{j,0.1} = p_j - \mathrm{softplus}(a_j)$$
$$\widehat{y}_{j,0.9} = p_j + \mathrm{softplus}(b_j).$$

Use the median as the point forecast. Train with masked quantile loss:

$$\mathcal{L} = \frac{\sum_{b,j} m_{bj} \sum_{\tau\in\{0.1,0.5,0.9\}} \rho_\tau(y_{bj} - \widehat{y}_{bj,\tau})}{3\sum_{b,j} m_{bj}}$$
$$\rho_\tau(r) = \max(\tau r, (\tau-1)r).$$

Here, m_{bj} marks valid targets. Use chronological splits with non-overlapping input/target windows. Compare station-wise MAE, flood errors and interval coverage against persistence. No results are claimed.

5. Operation and exceptional conditions.

- **Missing observations:** We will use the section 3's carry-forward method and train with sample missing data. Mark stale inputs. In case where repeated data is missing, will issue a labelled

persistence fallback with the latest reliable reading and its age.

- **Delayed observations:** Track measurement and arrival times. Use only arrivals received by t; add late readings at the next issuance. Reproduce delays during validation. Similar to how we worked in cases of missing data in MLOPs course.
- **Announced releases:** Align the latest plan announced by t to its effective times. Flag unknown values and exclude later revisions. Never substitute actual future releases for unavailable plans (A6).
- **(iv) Planned versus actual releases:** Flag measured departures from earlier plans. Use actual releases in history and train on recorded plan deviations. Update inputs at the next issuance.

Train offline. Every ten minutes, publish four forecasts with timestamps, intervals and quality flags. Eight short GRU streams and four queries over 294 tokens keep computation modest. Verify end-to-end latency below ten minutes before deployment; no benchmark is claimed.

6. Uncertainty, limitations, and additional information.

- **Ungauged tributary:** Sudden inflow may raise S3/S4 before inputs reveal it. Rainfall offers partial clues but not solid. Early surges may remain unpredictable for the model.
- **Variable travel times:** Floods or channel changes may invalidate learned lags. Monitor errors and coverage by flow regime; retrain after persistent drift.
- **Future rainfall and release deviations:** Unexpected storms or operations can cause large errors. Intervals depend on available information; coverage is not guaranteed.
- **Rare floods, faults and backwater:** Few extremes, faulty readings and backwater can undermine forecasts or A4. Apply quality checks and flood-focused validation. Calibrate nominal 80% intervals and check flood coverage separately.

The system remains usable without these additions, subject to the stated limits:

- **Gauge on the unmonitored tributary:** Earlier warning for S3/S4. Without it, the tributary blind spot remains.
- **Discharge, rating curves and channel geometry:** Better routing and backwater modelling. Without them, use learned level relationships with limited physical interpretation.
- **Catchment boundaries, more rain gauges and archived weather forecasts:** Better rainfall attribution and storm warning. Without them, retain R1/R2. Weather forecasts must predate or coincide with t.
- **Arrival/quality logs, longer flood records and archived plans:** Better validation and uncertainty estimates. Without them, simulate conservative delays, flag weak flood evidence and disable unverifiable plans. Calibration remains less certain.

Question 2 — Motion or Memory?

Protocol and verification

Moving MNIST [2] uses complete videos (`split=None`): train IDs 0–7999, validation 8000–8999 and test 9000–9999. Pixels are float32 in $[0, 1]$. Every learned model uses seeds 0/1, Adam at 10^{-3}, batch 32, 30 shuffled epochs and final weights. Each epoch samples one window per training video. Loss averages valid horizons within each video, then videos equally. A/B use normal speed; B adds uniform spatial symmetry. C/D first sample speed uniformly from $\{1, 2, 3\}$, then a valid start; neither adds spatial augmentation.

All models use five observations and a nine-step recurrent rollout: steps 1–5 consume observations; steps 6–9 consume predictions without detachment. Outputs 5–9 are the five forecasts. Invalid targets are masked. Unless stated otherwise, results use all 1,000 test videos at start $t = 1$; figures use seed 0. Learned table entries show mean (seed 0, seed 1). Errors have three significant figures; parameter counts are integers.

Hardware: Apple M5, macOS 26.6.2, MPS; PyTorch 2.14.1, torchvision 0.29.1, NumPy 2.5.4 and OpenCV 5.0.0. Training times average all 30 epochs. Inference times cover all 1,000 normal-speed videos, batch 32, with evaluation mode/no gradients and device synchronization; timing starts after device transfer. Wrapped timings include all eight passes and transforms. Wall-clock ratios can vary with system load.

The audit passed 155 checks and 36 independent CPU checkpoint-replay cases on 32 test videos. Maximum replay difference from saved MPS per-video scores was 1.27×10^{-7}. This is sampled learned-model verification. Full reference scores and all 950 flow pairs were independently reproduced.

Table 1: Transformation and window checks on the prescribed synthetic tensors.

Check	Expected	Yours
(a) M(M(x)) = x	0; torch.equal=True	0; torch.equal=True
(b) R^4(x) = x	0; torch.equal=True	0; torch.equal=True
(c) MRM(x) = R^-1(x)	0; torch.equal=True	0; torch.equal=True
(d) M(p)	(7, 10, 43)	(7, 10, 43)
(e) R(p)	(7, 43, 10)	(7, 43, 10)
(f) 1x input / target	([1, 2, 3, 4, 5], [6, 7, 8, 9, 10])	([1, 2, 3, 4, 5], [6, 7, 8, 9, 10])
(g) 2x input / target	([1, 3, 5, 7, 9], [11, 13, 15, 17, 19])	([1, 3, 5, 7, 9], [11, 13, 15, 17, 19])
(h) 3x input / target	([1, 4, 7, 10, 13], [16, 19])	([1, 4, 7, 10, 13], [16, 19])

Figure 1 — First 50 training videos, 950 consecutive frame pairs

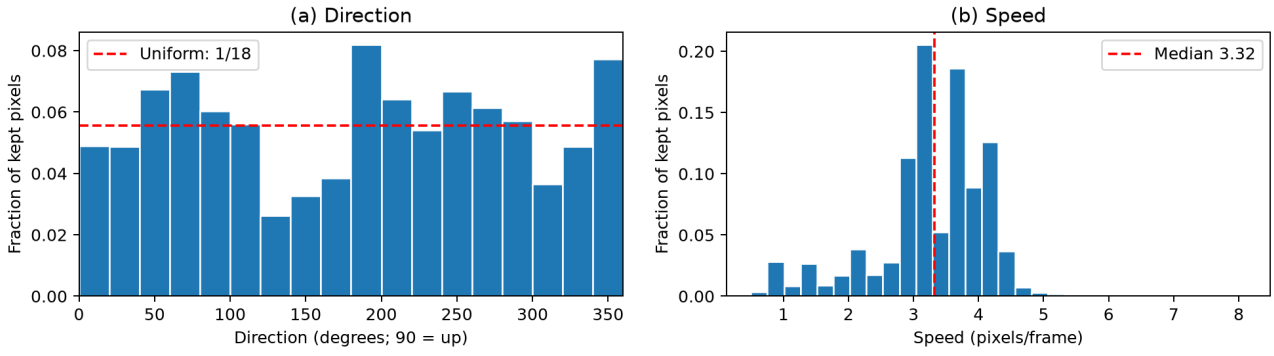


Figure 1: Training motion statistics: first 50 videos, 950 consecutive pairs and 103,050 retained pixels. Farneback parameters are $(0.5, 3, 15, 3, 5, 1.2, 0)$. Keep pixels above 0.2 in both frames and flow magnitude above 0.5. Direction bins span 20°; fractions sum to one. Median speed: 3.32 pixels/frame.

Q2.1

Figure 1 covers all directions, though frequencies are a little uneven but that can be ignored. Mirroring maps θ to $(180° - \theta)$ mod $360°$; rotation adds $90°$. Both of these transformations retain familiar motion directions, but can change their frequencies. Median speed is of 3.32 which implies nominal $2\times$‘/$3\times$ displacements of 6.64/9.95 pixels. Overlapping digits can corrupt optical flow, so these are approximate statistics and we try to minimize them by removing displacements mangintude of less than 0.5. The plot does not prove exact directional uniformity.

Q2.2

Mirror and rotation change digit appearance and motion direction wheras their speed magnitude remians the same. Speed-up changes displacement and temporal sampling but preserves digit appearance. These tests probe orientation transfer and speed familiarity under the same data and scoring rules.

Table 2: Normal-speed baseline errors on all 1,000 test videos.

Model	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$	Parameters
A	0.0170 (0.0176, 0.0164)	0.0204 (0.0208, 0.0200)	0.0228 (0.0231, 0.0225)	0.0247 (0.0249, 0.0245)	0.0265 (0.0265, 0.0265)	0.0223 (0.0226, 0.0220)	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

Each learned entry shows mean above (seed 0, seed 1). Parameters are single integer counts; all times are seconds.

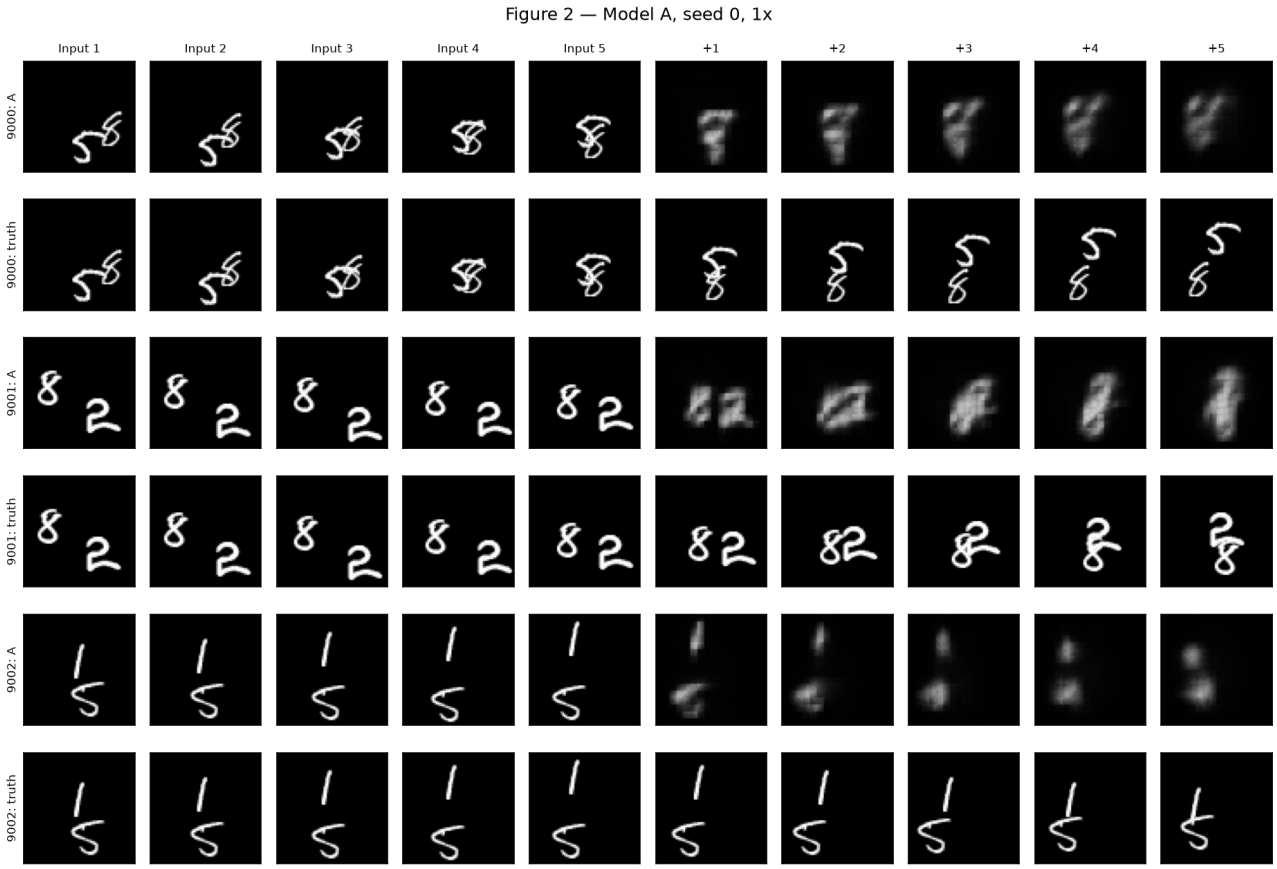


Figure 2: A at $1\times$, seed 0, test videos 9000–9002. Each pair of rows shows inputs followed by forecasts/truth. Video 9000 includes overlapping digits. Columns 6–10 are horizons 1–5.

Q2.3

A follows the ConvLSTM equations [1]. It folds $1 \times 64 \times 64$ into $16 \times 16 \times 16$, uses two biased 3×3 ConvLSTM layers of width 64, then a biased 64→16 head, pixel shuffle and sigmoid. States reset for each window.

One folded-grid step equals four original pixels. Current input and previous layer-1 hidden state cross two radius-one convolutions: $4(1 + 1) = 8$ pixels per axis. Previous layer-2 hidden state crosses one: 4 pixels. Thus the longest within-step reach is 8 per axis, or $8\sqrt{2}$ diagonally. Folding, the 1×1 head and unfolding add no centre-to-centre reach. Do not add patch width. Repeated recurrent steps extend paths over time.

Q2.4

At 1×, A's mean $E^{(5)} = 0.0223$ beats Blank (0.0426) and Copy-last (0.0641). Its horizon errors increases from 0.0170 to 0.0265. Figure 2 shows motion, but as seen from the images, overlap can merge diferent shapes and later forecasts blur or fade. Under uncertain future predictions, MSE favours averaged predictions. Moreover, low error itself does not mean proper and precise digit tracking.

Q2.5

I expect mirror and rotation errors to increase because digit appearance changes in this, even though Figure 1 contains motion in all directions. With A's 8-pixel per-axis reach and median speed 3.32, I beleive the model will not perform well at at 3× (9.95 pixels/frame), with possible 2× degradation (6.64) as well from unfamiliar motion.

Table 3: Normal-speed consistency gaps and transformed errors on all 1,000 test videos.

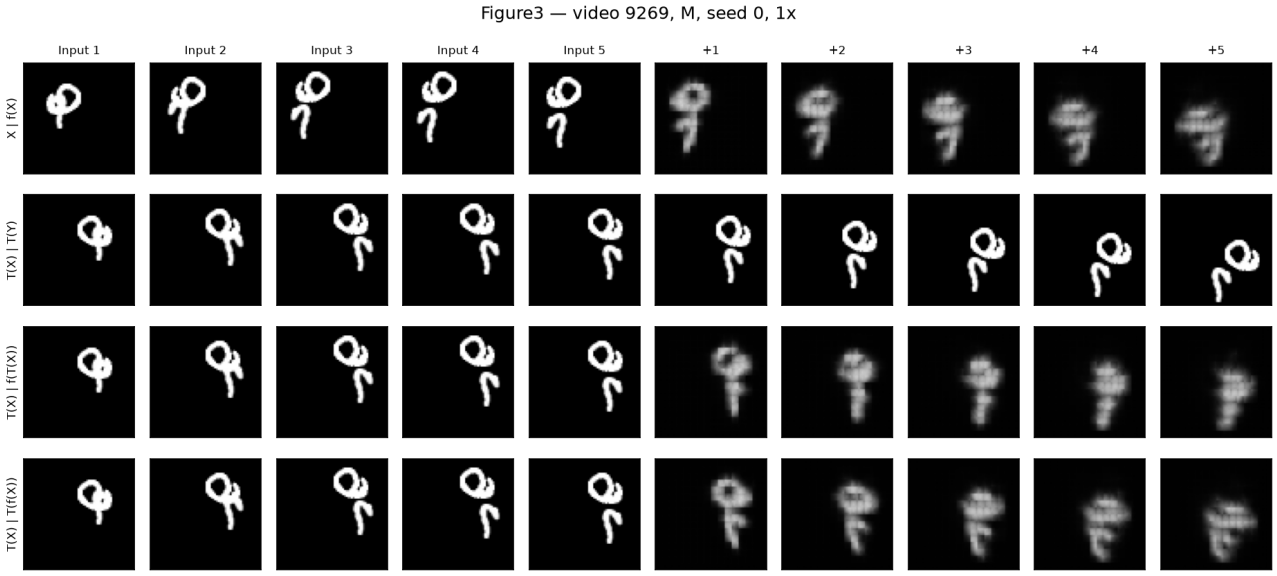
Model	$E^{(5)}$	C_M	E_M	C_R	E_R
A	0.0223 (0.0226, 0.0220)	0.00196 (0.00189, 0.00202)	0.0229 (0.0233, 0.0226)	0.00332 (0.00367, 0.00298)	0.0241 (0.0245, 0.0237)
Blank	0.0426	0.00×10^0	0.0426	0.00×10^0	0.0426
Copy-last	0.0641	0.00×10^0	0.0641	0.00×10^0	0.0641

Each learned entry shows mean above (seed 0, seed 1). Parameters are single integer counts; all times are seconds.

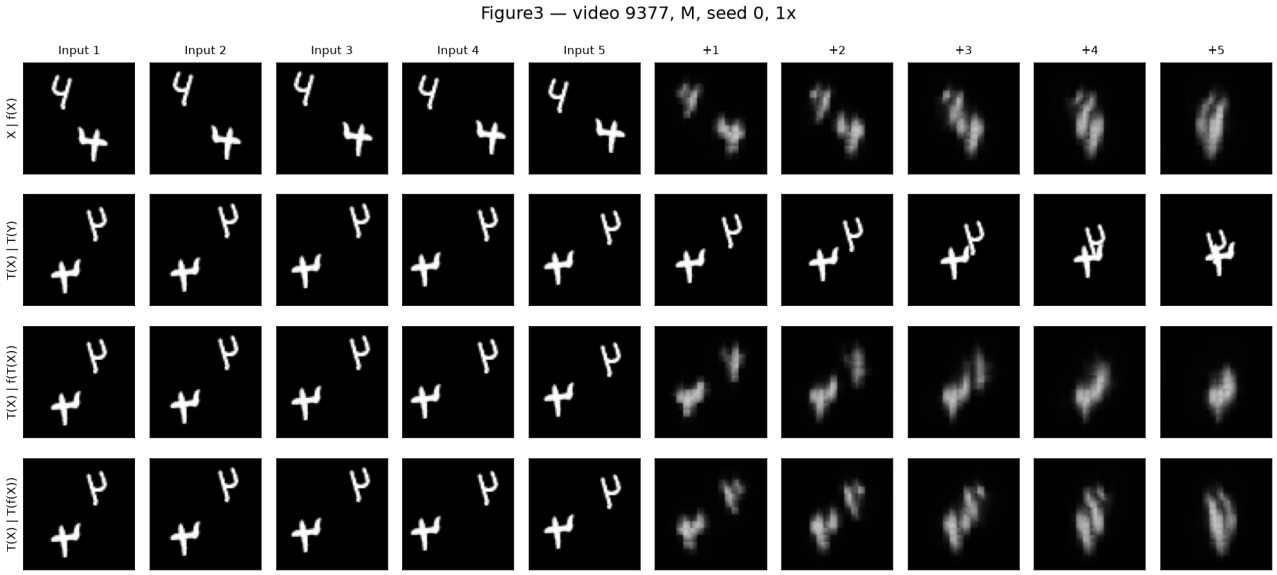
Q2.6

Blank commutes with every spatial transform. Copy-last also commutes because it repeats the transformed last input. Both have exactly zero gaps, yet transformed errors remain 0.0426 and 0.0641. Consistency alone is insufficient. That is why we must always pair C_T with E_T.

Figure 3: A consistency failures, 1×, seed 0. Rows: $X|f(X)$, $T(X)|T(Y)$, $T(X)|f(T(X))$, $T(X)|T(f(X))$. Five inputs precede five horizons.

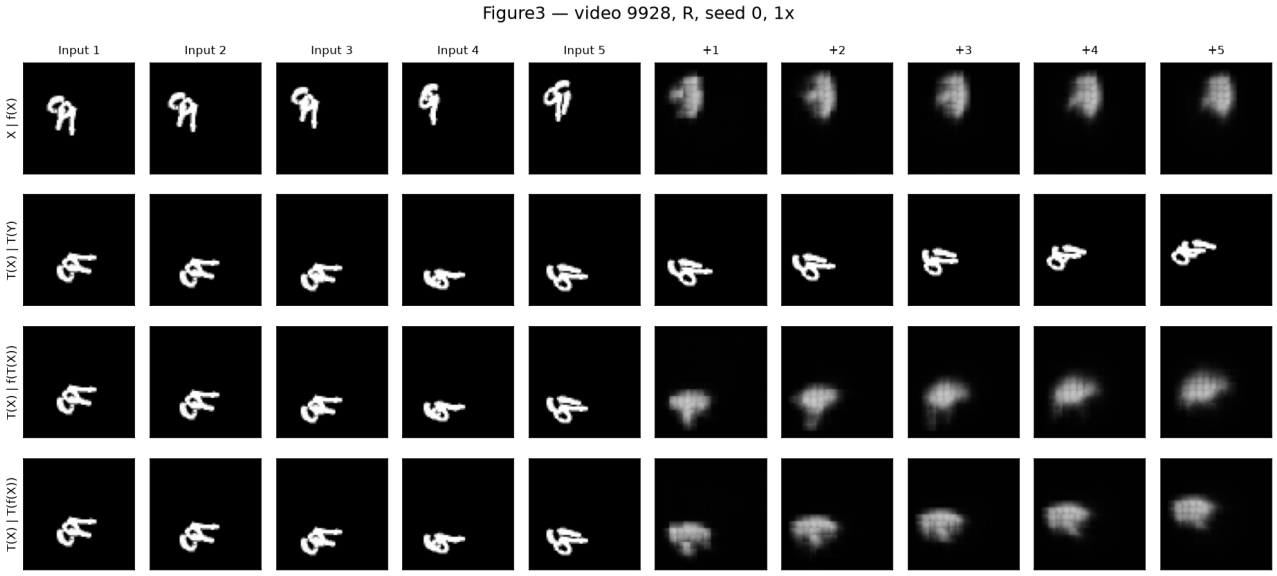


(a) Video 9269, M. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure3_M_video9269.gif`.

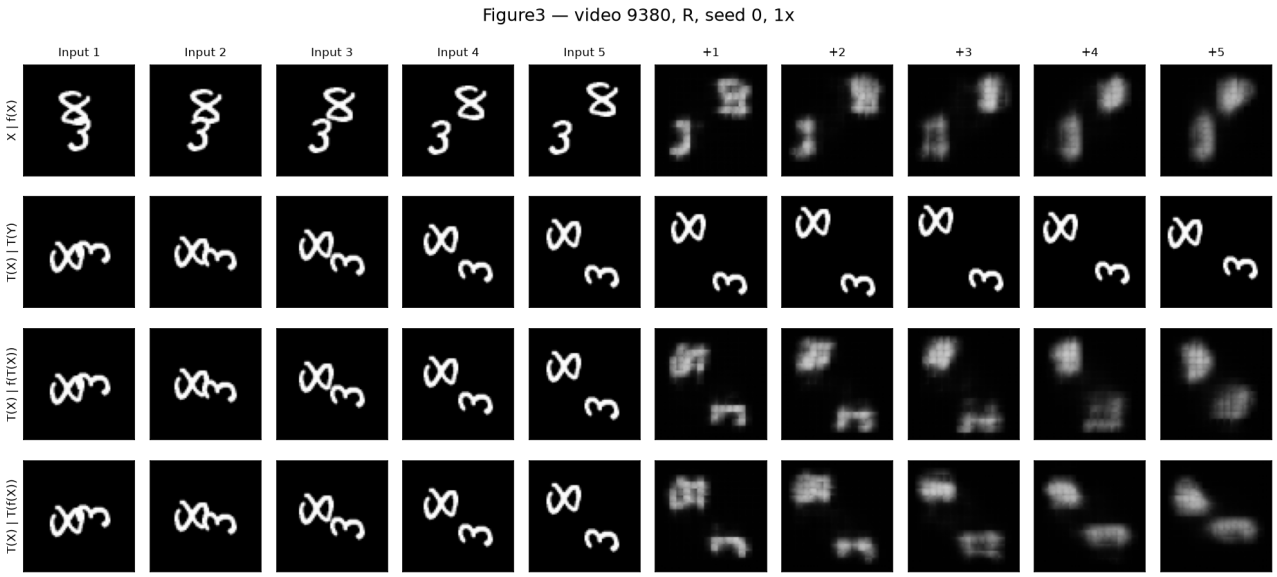


(b) Video 9377, M. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure3_M_video9377.gif`.

Figure 3: A consistency failures, 1×, seed 0. Rows: $X|f(X)$, $T(X)|T(Y)$, $T(X)|f(T(X))$, $T(X)|T(f(X))$. Five inputs precede five horizons. Continued.



(c) Video 9928, R. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure3_R_video9928.gif`.



(d) Video 9380, R. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure3_R_video9380.gif`.

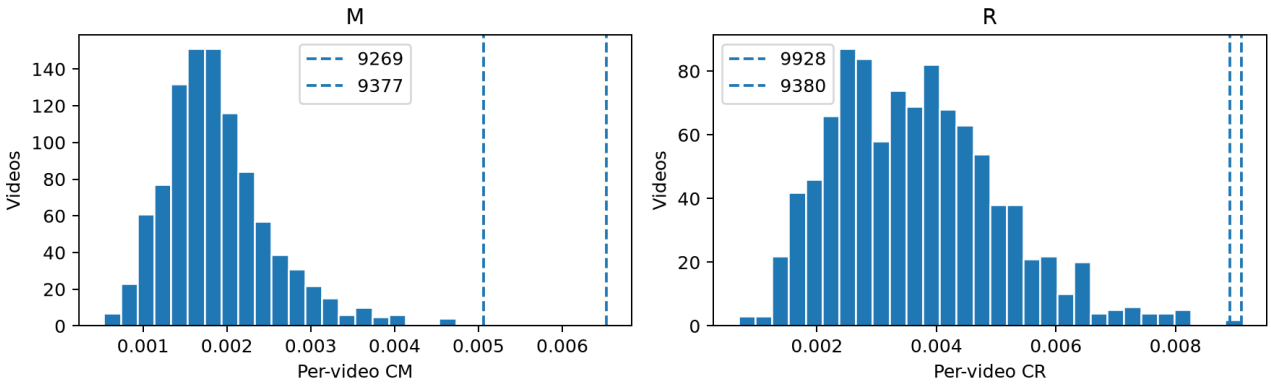


Figure 4: A, seed 0, at 1×: per-video mirror/rotation consistency gaps for all 1,000 test videos. Dashed lines mark the exact Figure 3 selections; their transformed errors accompany the gaps in Table 3 and the example discussion.

Q2.7

For video 9928, rotation R, A at 1×/seed 0 gives different paths in Figure 3: row 3's diffuse prediction shifts up/right, while row 4 rises faster with little horizontal movement. The difference persists across horizons. Its per-video $C_R = 0.00910$, with $E_R = 0.0236$. I retain this video and transform for Q2.16.

Q2.8

Rotation is harder for A at 1×: $(C_R, E_R) = (0.00332, 0.0241)$ versus mirror $(0.00196, 0.0229)$; original $E^{(5)} = 0.0223$. Figure 1 already contains broad direction coverage, whereas transformed digit shapes differ. Appearance sensitivity and absent built-in equivariance are plausible causes; unequal direction frequencies prevent a clean appearance-only attribution.

Table 4: Speed tests on all 1,000 test videos. Only the first two horizons support all-speed comparisons.

Model	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	$E^{(5)}$ 1×	$E^{(5)}$ 2×
A	0.0187 (0.0192, 0.0182)	0.0445 (0.0445, 0.0446)	0.0522 (0.0527, 0.0518)	0.0223 (0.0226, 0.0220)	0.0496 (0.0501, 0.0490)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

Each learned entry shows mean above (seed 0, seed 1). Parameters are single integer counts; all times are seconds.

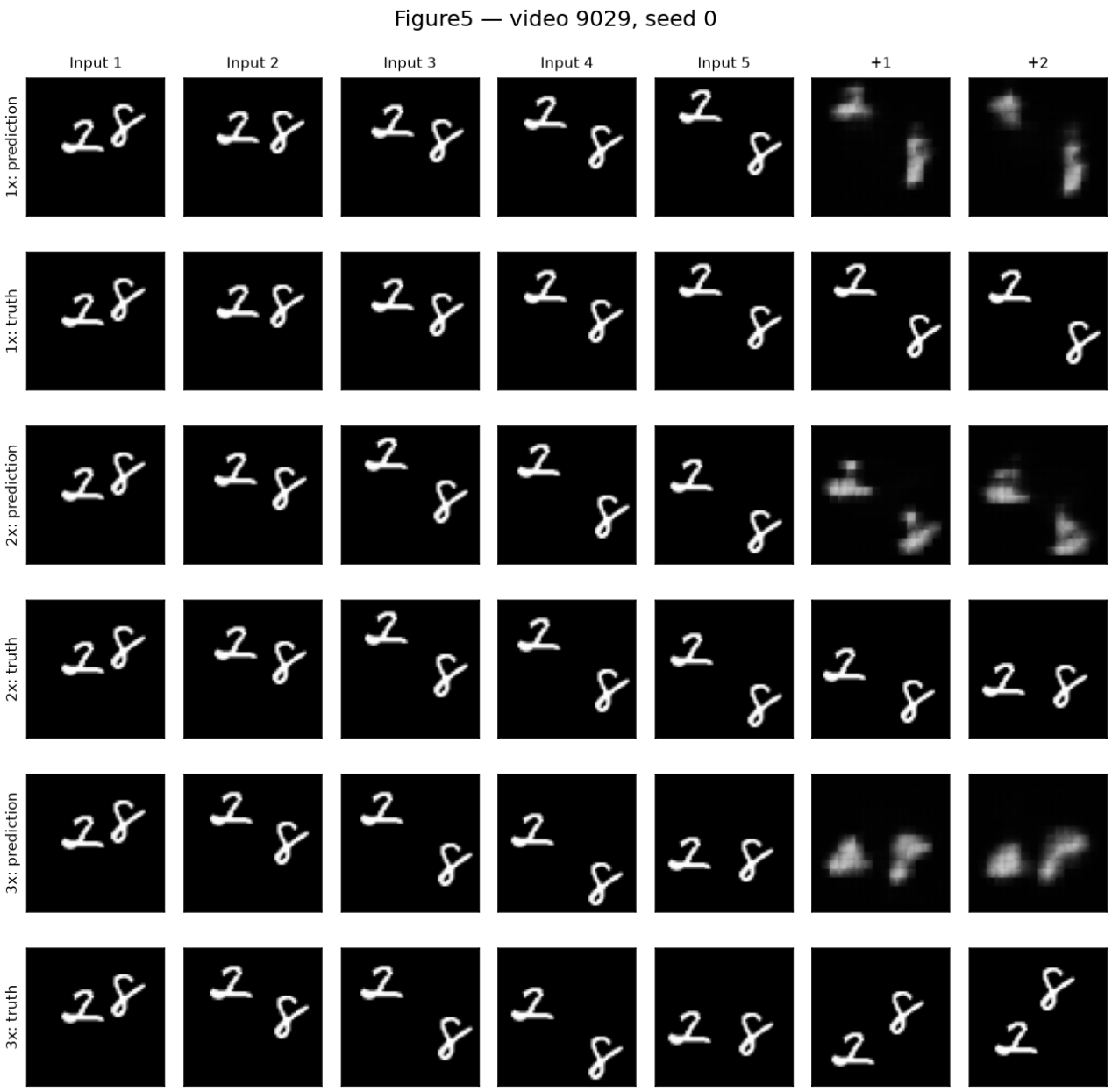


Figure 5: A, seed 0, video 9029: five inputs and two forecast/truth horizons at each speed. Digits do not overlap in the displayed true frames. The final model reuses this video in Figure 6(b).

Q2.9

A's mean $E^{(2)}$ rises from 0.0187 at 1× to 0.0445/0.0522 at 2×/3×. Figure 5 shows misplaced, blurred forecasts and fading at higher speed. Degradation already occurs at 2×, below the nominal 8-pixel reach comparison. Flow speed is a magnitude, but reach is per axis; 9.95 pixels at 3× need not exceed an 8-pixel component bound. Reach also omits appearance, velocity estimation, bounces and optimization. It is not an exact failure-speed threshold.

Q2.10

A's results alone cannot separate unfamiliar speed from limited reach. Compare A with C under the same architecture, then C with a larger-reach D under the same speed-augmented protocol. A stronger follow-up would vary reach alone at comparable parameter count and optimization, with candidate selection on validation.

Q2.11

I expect B to reduce mirror and rotation failures through symmetry augmentation. C should also improve 2×/3× forecasts through speed augmentation, but cannot increase architectural reach. I think only D will be able to handle this faster motion better than C through larger reach, although blur and shape errors may remain

Table 5: Controlled experiments on all 1,000 test videos. D includes second-layer gate normalization.

Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Parameters	Train s/epoch	Inference s / 1,000
A	0.0223 (0.0226, 0.0220)	0.00196 (0.00189, 0.00202)	0.0229 (0.0233, 0.0226)	0.00332 (0.00367, 0.00298)	0.0241 (0.0245, 0.0237)	0.0187 (0.0192, 0.0182)	0.0445 (0.0445, 0.0446)	0.0522 (0.0527, 0.0518)	480,784	35.6 (32.9, 38.4)	1.05 (1.06, 1.05)
B	0.0227 (0.0232, 0.0222)	0.00107 (0.00112, 0.00103)	0.0227 (0.0232, 0.0221)	0.00131 (0.00137, 0.00126)	0.0227 (0.0233, 0.0222)	0.0190 (0.0193, 0.0186)	0.0449 (0.0441, 0.0458)	0.0530 (0.0514, 0.0545)	480,784	39.2 (39.5, 38.8)	1.13 (1.39, 0.866)
C	0.0274 (0.0283, 0.0265)	0.00101 (9.75×10^{-4}, 0.00104)	0.0275 (0.0284, 0.0266)	0.00207 (0.00170, 0.00243)	0.0284 (0.0291, 0.0277)	0.0238 (0.0246, 0.0230)	0.0290 (0.0294, 0.0286)	0.0336 (0.0339, 0.0333)	480,784	30.6 (31.2, 30.0)	0.822 (0.832, 0.812)
D	0.0195 (0.0196, 0.0194)	0.00295 (0.00312, 0.00278)	0.0206 (0.0206, 0.0205)	0.00419 (0.00424, 0.00415)	0.0219 (0.0220, 0.0218)	0.0161 (0.0162, 0.0160)	0.0222 (0.0223, 0.0222)	0.0275 (0.0276, 0.0273)	721,024	102 (100, 105)	2.56 (2.57, 2.55)

Each learned entry shows mean above (seed 0, seed 1). Parameters are single integer counts; all times are seconds.

Q2.12

D keeps folding factor 4 and the same rollout, but uses width 54, dense 3×3/5×5 ConvLSTM layers and a 54→16 head. Its second layer normalizes 216 gate preactivations across channels at each spatial position before gate nonlinearities. A–C remain unchanged. No attention is used in D.

Current input or previous layer-1 state crosses radii 1 and 2: $4(1+2) = 12$ pixels per axis. Previous layer-2 state reaches 8. The diagonal bound is $12\sqrt{2}$. Channel normalization adds no spatial reach. Parameters are $136{,}296 + 583{,}416 + 880 + 432 = 721{,}024 \leq 721{,}176$. Thus D has strictly greater reach than A within the cap.

Q2.13

Orientation/appearance: B lowers the 1× mirror gap/error to $(0.00107, 0.0227)$ and rotation to $(0.00131, 0.0227)$, which supports an orientation-transfer problem. It does not establish appearance as the sole cause.

Unfamiliar motion: C retains A's reach but reduces 2×/3× $E^{(2)}$ by 34.9%/35.7%. Speed exposure therefore explains part of A's failure.

Architecture: D further lowers C's 2×/3× errors from 0.0290/0.0336 to 0.0222/0.0275. This supports D's architectural package. Kernels, width and normalization all differ, so reach alone is not isolated. C improves at 3× but still trails D. More data can improve use of existing paths; it cannot create absent within-step connections.

Q2.14

At original 1×, B increases A's mean $E^{(5)}$ by 1.79%, and C by 22.7%. C trades normal-speed accuracy for stronger fast forecasts. D instead improves A's normal-speed error by 12.4% and also improves fast performance, at higher training/inference cost. That trade-off is useful when fast-motion accuracy matters.

Table 6: Symmetrised models on all 1,000 test videos. Sym-D is the final model; training time is that of its underlying model.

Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Parameters	Train s/epoch	Inference s / 1,000
Sym-B	0.0222 (0.0227, 0.0217)	7.82×10^{-17} (7.25×10^{-17}, 8.39×10^{-17})	0.0222 (0.0227, 0.0217)	6.29×10^{-17} (5.86×10^{-17}, 6.72×10^{-17})	0.0222 (0.0227, 0.0217)	0.0185 (0.0189, 0.0182)	0.0441 (0.0435, 0.0447)	0.0520 (0.0507, 0.0532)	480,784	39.2 (39.5, 38.8)	7.24 (7.18, 7.30)
Sym-D	0.0194 (0.0194, 0.0193)	9.36×10^{-17} (9.73×10^{-17}, 8.99×10^{-17})	0.0194 (0.0194, 0.0193)	7.44×10^{-17} (7.74×10^{-17}, 7.14×10^{-17})	0.0194 (0.0194, 0.0193)	0.0159 (0.0160, 0.0158)	0.0218 (0.0219, 0.0218)	0.0269 (0.0270, 0.0267)	721,024	102 (100, 105)	20.6 (20.8, 20.4)

Each learned entry shows mean above (seed 0, seed 1). Parameters are single integer counts; all times are seconds.

Q2.15

Choose Sym-D as the final model; no optional D+B is trained. Define

$$\bar{f}(X) = \frac{1}{8} \sum_{g \in G} g^{-1} f(gX).$$

For any $h \in G$, set $k = gh$:

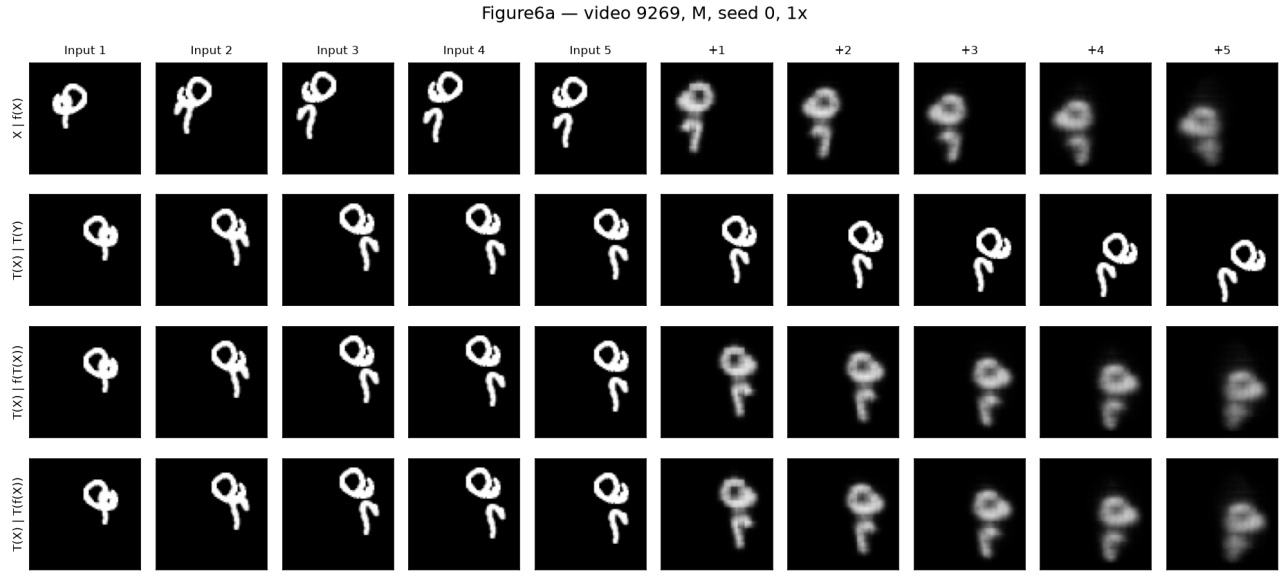
$$\bar{f}(hX) = \frac{1}{8} \sum_{k \in G} hk^{-1} f(kX) = h\bar{f}(X).$$

The group action distributes over averaging. Thus mirror/rotation gaps are exactly zero in real arithmetic, for any underlying f; the measured gaps reflect floating-point order.

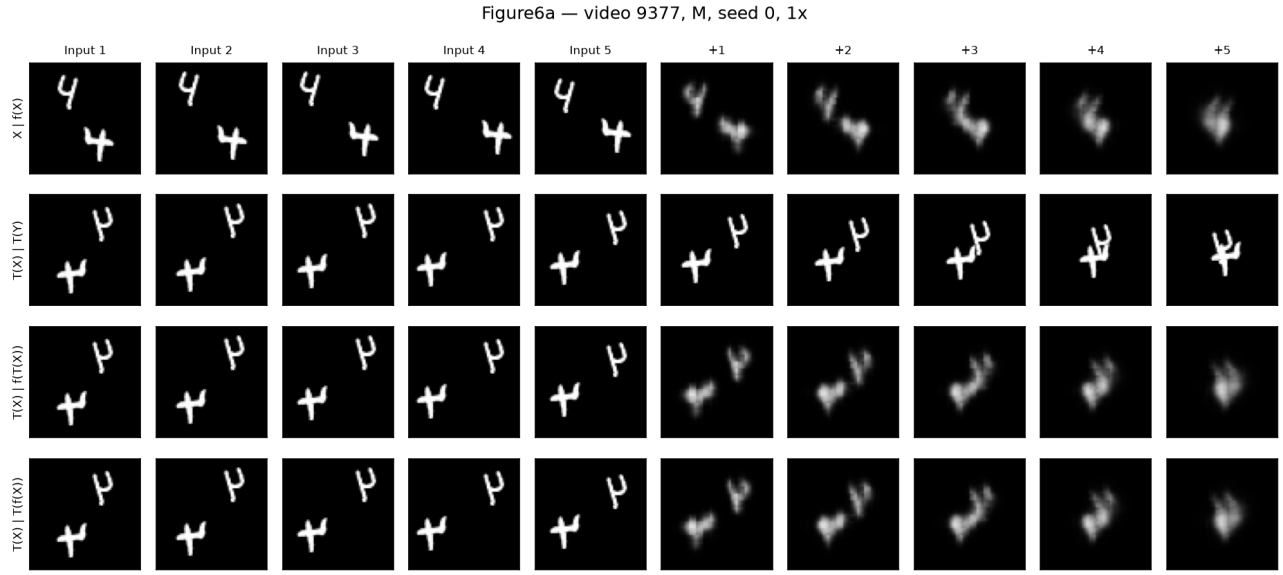
B learns approximate symmetry; its paired gaps/errors remain those in Table 5. Sym-B gives $(C_M, E_M) = (7.82 \times 10^{-17}, 0.0222)$ and $(C_R, E_R) = (6.29 \times 10^{-17}, 0.0222)$. Sym-D gives $(9.36 \times 10^{-17}, 0.0194)$ and $(7.44 \times 10^{-17}, 0.0194)$. Bare D has larger gaps than A despite lower transformed errors.

Sym-D improves D's mean normal $E^{(5)}$ by 0.874%; Sym-B improves B by 2.39%. Averaging can blur disagreeing forecasts or worsen individual videos: D's original $1\times$ error for video 9269 rises from 0.0274 to 0.0287 after wrapping. There is no aggregate normal-speed penalty here. Inference costs rise from 1.13 to 7.24 seconds for B and 2.56 to 20.6 for D; the wrapper performs eight passes, adds no parameters and needs no retraining.

Figure 6: (a) Final Sym-D, seed 0, on the exact Figure 3 videos/transforms. Four rows and ten columns retain the original layout.

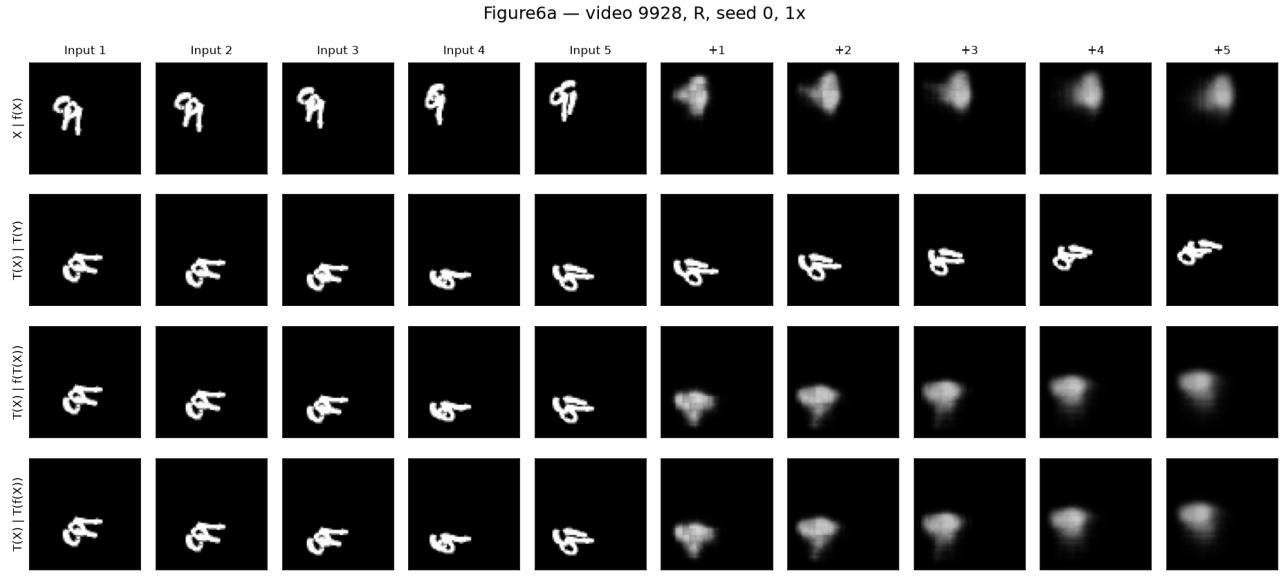


(a.1) Video 9269, M. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure6a_M_video9269.gif`.

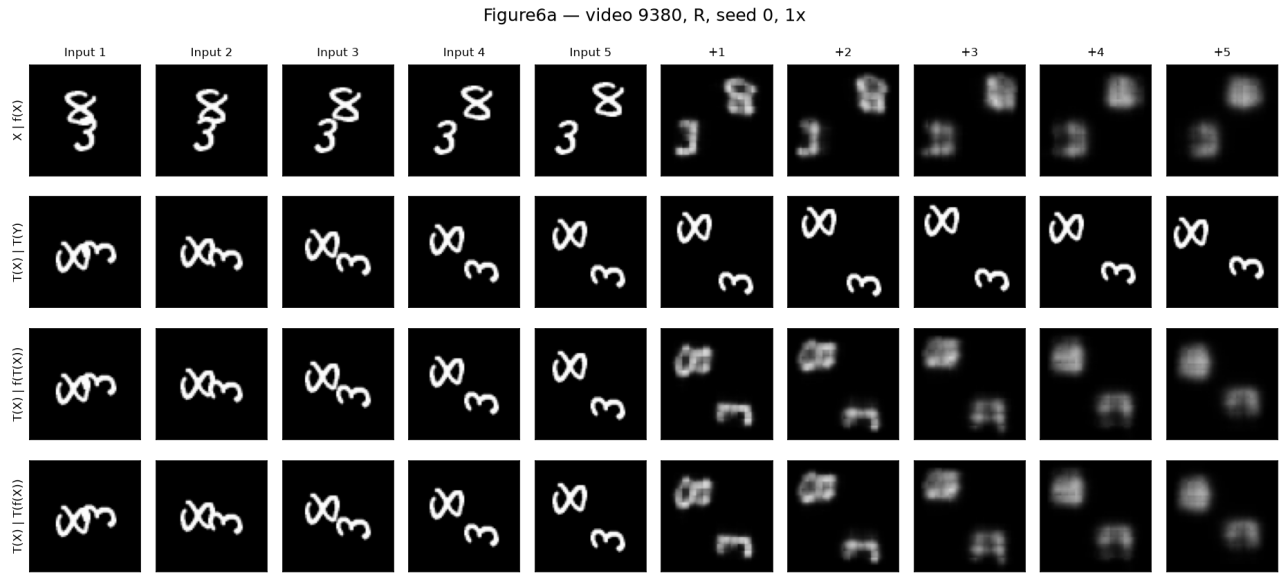


(a.2) Video 9377, M. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure6a_M_video9377.gif`.

Figure 6: (a) Final Sym-D, seed 0, on the exact Figure 3 videos/transforms. Four rows and ten columns retain the original layout. Continued.



(a.3) Video 9928, R. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure6a_R_video9928.gif`.



(a.4) Video 9380, R. Five-horizon GIF: `Question2_normalized/artifacts/gifs/Figure6a_R_video9380.gif`.

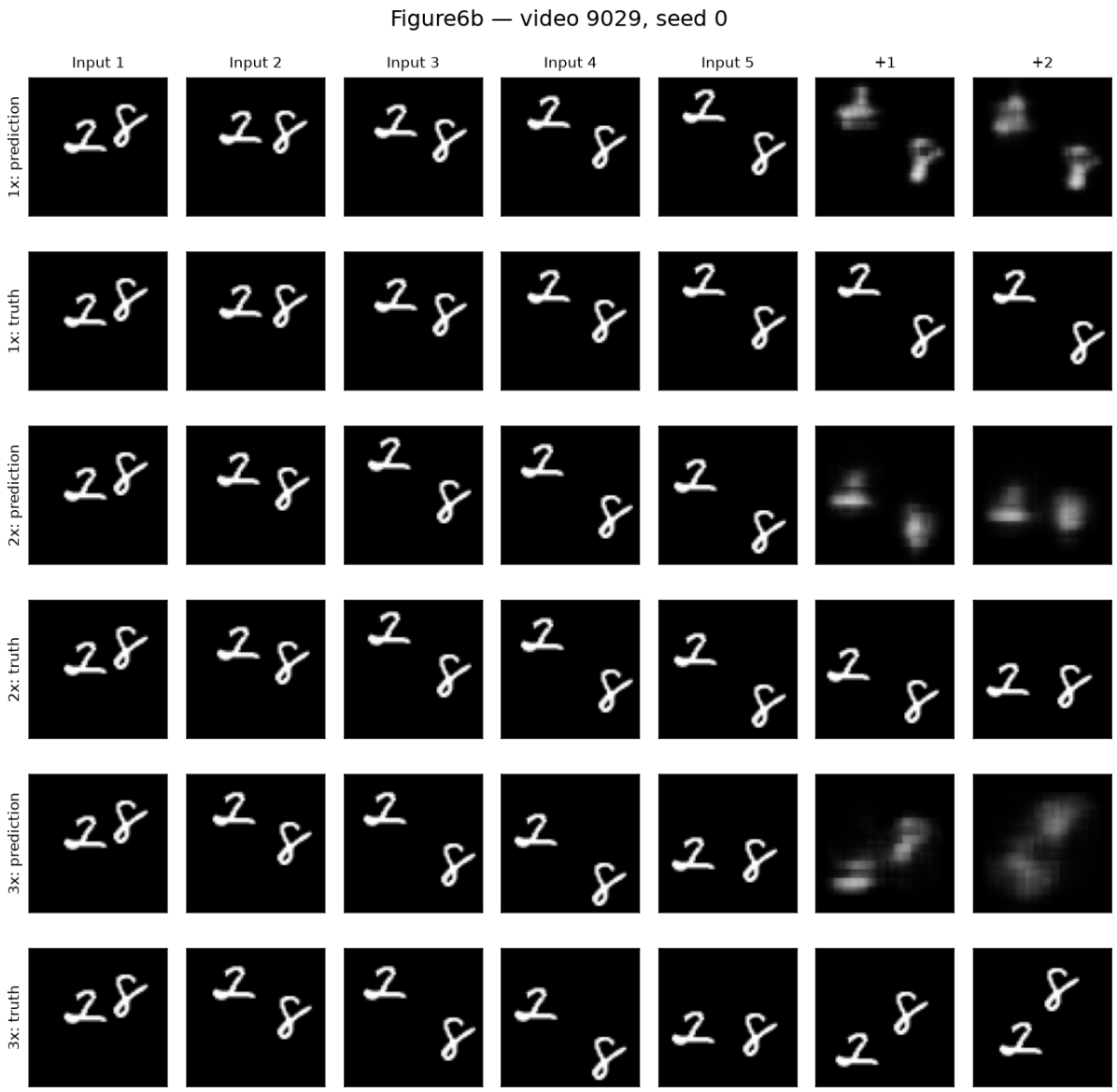


Figure 6: (b) Final Sym-D, seed 0, on the exact Figure 5 video 9029. Five inputs and two forecast/truth horizons appear at each speed.

Q2.16

For the same video 9928, R, $1\times$/seed 0, Sym-D makes rows 3/4 agree: $(C_R, E_R) = (8.27\times10^{-17}, 0.0142)$, versus A's $(0.00910, 0.0236)$. Bare D already reduces them to $(0.00617, 0.0169)$. D's learned architecture improves prediction; wrapping removes orientation inconsistency. Neither guarantees correct digit shapes: Figure 6(a) still shows blur and fading. Figure 6(b) also improves fast forecasts for video 9029, but does not reproduce sharp truth.

Q2.17

The final Sym-D model learned useful motion cues. Across all test videos and both seeds, its mean $E^{(5)}$ is 0.0194, below A's 0.0223 and both deterministic references. Its $2\times/3\times$ $E^{(2)}$ values are 0.0218/0.0269, versus A's 0.0445/0.0522. The required examples show moving predictions, but blur and fading remain. Near-zero symmetry gaps prove equivariance, not correct motion. A stronger test would use independently generated trajectories and digit shapes. The fixed test set was already seen during earlier model studies, so it is not an untouched holdout for the full development history.

Study provenance. Earlier compliant D variants performed worse; one unnormalized seed collapsed. The current D-only gate-normalization fix passed separate training/validation pilots before these final runs. Current C/D validation diagnostics preceded current D test evaluation. Earlier test exposure remains part of the study history; no failed seed was dropped and no best-validation checkpoint replaced final weights.

References

[1] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W. Woo. Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting. NeurIPS, 2015.

[2] N. Srivastava, E. Mansimov, and R. Salakhutdinov. Unsupervised Learning of Video Representations using LSTMs. ICML, 2015.

Generative AI assistance disclosure

AI was used in understanding the complete intuition of the assignment and how the different tasks led to a bigger picture. It helped me with the code as well. AI was further used to help me diagnose an issue in which my model D seed 0 gave blank aoutputs. The issue turned to be a missing normalization layer. The report data was compiled and fornmatted by AI. In some cases where I couldnt understand the results/outputs, AI was used to help me understand the results.